OpenSAI 下一版本开发计划 • 讲稿（汇报用）

2026 年 6 月

目录

M1 • OpenClaw 插件 MVP（最高优先，跑在现有栈上） 1
任务分解 1
M2 • Tailscale 跨网组网 3
任务分解 3
M3 • 跨平台设备端 4
任务分解 4

用法：标题与内容均**与官方版《OpenSAI 下一版本开发计划》一字对应**；每一步下面用**方框**给出一两句口述讲稿，汇报时照着念即可。标 [已具备基础] 的，是在现有能力上扩展。

M1 · OpenClaw 插件 MVP（最高优先，跑在现有栈上）

目标：以原生 OpenClaw AgentSkill 形式暴露 OpenSAI 能力，让 Agent 用**已注册算法**编排 DAG、卸载执行、取回结果；在**现有 Windows/桌面单机或局域网栈**上端到端跑通；对接 Agent Control，守住 “仅预置算法” 边界。

> **讲稿** | 把 OpenSAI 接到 AI 助手上，让 AI 用自然语言就能指挥设备群跑任务、取结果；现有 Windows 栈就能做，红线是只能用审核过的算法。

任务分解

- **M1.1 抽取 OpenSAI 客户端 SDK（`opensai_client`）** [已具备基础]：把现有 `inject_collab` / `user_client` 里的 “提交 DAG + 等结果” 逻辑抽成一个**框架无关**的可复用 Python 模块（封装控制面提交、dagResult 等待、设备/监控查询）。插件与未来其他调用方都复用它，避免重蹈 naive TCP framing / user_beat 路由的坑。**注**：目前提交逻辑已在 `inject_collab` / `run_tests` / `test_full` / `test_bridge` / `user_client` 多处各自重复实现一遍——抽 SDK 时应一并收编这些重复实现，同时作为回归测试的统一客户端基座。

> **讲稿** | 把现在分散在好几处、各写一遍的 “提交任务＋等结果” 代码，收拢成一个统一可复用的客户端模块；已有现成实现，主要是收编统一。

- **M1.2 定义 Agent 工具集（tool schema）** [已具备基础]：以下工具的**数据源均已就绪**，Web 桥接也已以 HTTP 端点暴露（`/api/algorithms`、`/api/devices`、`/api/events`、`/api/dag/upload`），本任务主要是在其上封一层稳定、Agent 友好的工具 schema（直连或经桥接均可）：
  - `list_algorithms()` —— 返回已注册算法目录（名称、参数、是否产出/是否需输入），数据来自 `device/implements/Algorithm_*`（桥接 `/api/algorithms` 已可用）。
  - `list_devices()` / `cluster_status()` —— 读 Redis `OPENSAI_MONITOR_VIEW`（T1.3 聚合视图：设备、槽位、负载、账本、链路；桥接 `/api/devices` 已可用）。
  - `submit_dag(dag_spec)` —— 校验后提交，返回 `job_id`。
  - `get_result(job_id)` / `wait_result(job_id, timeout)` —— 轮询/等待 dagResult。
  - `submit_task(algorithm, params)` —— 单节点便捷封装（可选）。

讲稿 | 给 AI 定义一组标准工具：列算法、看设备、提交任务、取结果；这些数据源和接口都已就绪，只需封一层。

- **M1.3 Agent 友好的 DAG 描述 schema**：结构化 JSON（节点：id / algorithm / params / beforeNodes-afterNodes / 可选资源提示 cores • bw • mem / 可选 directAllocationId），与现有 `dagData` 对齐但更易由 Agent 生成。

讲稿 | 定一种 AI 容易生成、不易填错的任务描述格式（每步用什么算法、什么参数、依赖谁）。

- **M1.4 “仅预置算法” 边界校验**：`submit_dag` 校验 ① 只引用已注册算法 ② 无环 ③ 参数合法；非法（未知算法/任意代码）一律拒绝。**服务端二次校验**（纵深防御），不只靠插件层。**现状**：设备端已对未知/不可用算法做**执行期**容错（回 ERROR 结果、不再挂起、放置预留正常释放）；本任务要补的是更靠前的**提交期拒绝**（在 `submit_dag` 与服务端入口处，未知算法/成环/非法参数直接拒收，不进入派发）。

讲稿 | 安全关卡：只放行审核过的算法，提交时和服务端各校验一遍；未知算法现在已能安全报错，这步把拦截提前到一提交就拒。

- **M1.5 长任务的同步/异步处理**：卸载是长耗时操作 → 用 `job_id` + 轮询（`get_result`）或带超时的 `wait_result`，适配 Agent 工具调用的超时模型。

讲稿 | 任务耗时长，改成 “下单给号、凭号取结果” 的异步方式，适配 AI 工具的超时限制。

- **M1.6 对接 OpenClaw Agent Control**：把 OpenSAI 工具注册到 Agent Control；按策略放行/拦截（允许哪些工具、频率限制）；“仅预置算法” 边界同时由 M1.4 校验与 Agent Control 策略双重保证。

讲稿 | 把这些工具接入 AI 平台的权限系统，统一管控谁能用、用多频，能拦下被禁止的操作。

- **M1.7 打包为原生 AgentSkill**：`skills/opensai/` 自包含 Python 模块（清单/元数据、自动发现）、配置（OpenSAI server/Redis 地址）、依赖声明。

讲稿 | 打包成一个即插即用的插件，配好服务器地址，装上就能用。

- **M1.8 端到端 demo**：Agent 用自然语言描述一个感知/计算任务 → 插件构造预置算法 DAG（如协同感知 FeatureExtract+Fusion，或 ProduceBlob→ConsumeBlob）→ 卸载到设备群 → 返回结果（现有 Windows 栈）。

讲稿 | 做一个端到端演示：AI 听一句话，自动编排任务、卸载执行、返回结果，证明整条链路打通。

- **M1.9 测试**：插件单测（schema 校验、非预置算法拒绝）、集成测（插件 → 栈 → 结果）并入回归。

讲稿 | 给插件配自动化测试并入回归，防止以后改动悄悄把它弄坏。

M2 · Tailscale 跨网组网

目标：让不同 LAN/NAT 下的设备经一个 Tailscale tailnet 加入同一 OpenSAI 集群——控制面（设备 ↔server TCP）与数据面（设备 ↔ 设备 QUIC）都在 tailnet 上工作——使 M1 的插件能把任务分发到**真正跨网**的设备。各端安装官方 Tailscale 客户端。

讲稿 | 用 Tailscale 给不同网络的设备架一条加密专线，让它们跨网也能像在同一局域网一样协作，M1 的插件就能调度到跨网设备。

任务分解

- **M2.1 tailnet 建立与设备纳管**：开通 tailnet；基于 auth key 的设备接入（ephemeral/tagged key）；各 OS 安装文档（Win/Linux/macOS；Android 客户端供瘦设备用）。

讲稿 | 开通专线、发接入凭证、写各系统的接入文档，让设备一键加入。

- **M2.2 地址/配置改造**：把硬编码的 `127.0.0.1`/LAN 地址改为 tailnet 地址；server 公布其 tailnet IP / MagicDNS 名；设备注册其 tailnet IP（大脑已跟踪设备 IP/端口用于 D2D，指向 tailnet IP 即可）。新增配置：`OPENSAI_SERVER_ADDR`、设备数据端口经 tailnet 可达。

讲稿 | 把写死的本机/局域网地址改成专线地址，让大脑和设备走专线互通。

- **M2.3 服务发现**：server 经 MagicDNS（如 `opensai-server.<tailnet>`）发现，或保留协调器/Redis 注册表但填入 tailnet 地址；D2D QUIC 用注册表中的 tailnet IP 寻址。

讲稿 | 让设备能自动发现大脑服务器在专线上的地址，不用人去配。

- **M2.4 QUIC-over-WireGuard 验证** [已具备基础]：验证 aioquic D2D 经 tailnet（UDP 走 WireGuard）可用；测 MTU/吞吐影响；评估 aioquic ~10MB/s 上限 + WireGuard 开销叠加并调优。**注**：本版**已实现多链路数据面（中继 + 多网卡并行聚合）**，可直接用于缓解单链路带宽上限、并天然适配 DERP 中继路径，跨网时复用即可。

讲稿 | 验证设备间数据传输在专线上的可用性和吞吐；已有的多链路能力可直接用来缓解带宽瓶颈。

- **M2.5 连通性鲁棒性** [已具备基础]：处理直连 vs DERP 中继回退（无直连路径时 Tailscale 走中继）及其对时延/吞吐的影响——**大脑放置已是链路感知**（按各设备实测链路带宽 `observed_links` 估传输代价选设备，见 `offloading_brain._transfer_seconds`），跨网只需让 `observed_links` 采到 tailnet/DERP 实测速率即自动生效；另处理设备 IP 变化、断线重连、睡眠/唤醒、离线。

讲稿 | 处理连不上时走中继的兜底和断线重连等；放置已是链路感知，跨网采到真实网速就自动选更快的路。

- **M2.6 安全/ACL**：tailnet ACL（哪些设备可达 server 控制端口、哪些可 D2D）；设备身份/标签；与 M1 的 Agent 边界打通（Agent 提交的任务只在被授权设备上跑）；OpenSAI 端口仅绑 tailnet 接口。

讲稿 | 配安全规则，限定谁能连控制面/数据面，并与 M1 的算法边界打通；服务只在专线上开。

- **M2.7 测试**：跨网集成测（≥2 台处于不同网络的机器，或模拟）经 tailnet 接入并跑通一个 DAG；验证 D2D 传输经 tailnet。

讲稿 | 用两台不同网络的机器实测，验证跨网接入、跑通任务、数据确实走专线。

M3 · 跨平台设备端

目标：设备端在 Win/Linux/macOS 作一等计算节点运行，在 Android 作**瘦客户端**（采集/轻任务/结果展示，不跑重模型），经一个平台抽象层统一。

讲稿 | 让设备端在 Win/Mac/Linux 当全力节点、手机当瘦客户端，经一个平台抽象层统一，大脑按设备能力派活。

任务分解

- **M3.1 平台差异盘点 + 抽象层**：定义 `PlatformAdapter` 接口统一资源控制——绑核、限内存、进程池模型、BLAS 线程对齐（已有 threadpoolctl 基础）。实现：Windows（Job Object + cpu_affinity）、Linux（RLIMIT_AS + cpu_affinity）、macOS（**无 cpu_affinity → 优雅退回线程数限制**；RLIMIT_AS 有局限）、Android-thin（最小/无）。

讲稿 | 做一个平台适配层，把各系统在绑核/限内存/进程池上的差异统一掉（Mac 无绑核就退化为限线程数）。

- **M3.2 macOS 一等计算端**：处理 cpu_affinity 缺失（已有部分 fallback）、fork-vs-spawn 进程池启动可靠性、全栈在 macOS 跑通、打包（venv 引导 / py 启动器）。

讲稿 | 让全栈在 macOS 上稳定跑通并打包，处理它的进程启动与资源限制差异。

- **M3.3 Linux 一等计算端**：验证（最接近现状）；RLIMIT_AS 内存上限；cpu_affinity；打包。

讲稿 | 在 Linux 上验证内存上限、绑核并打包（最接近现状）。

- **M3.4 Android 瘦客户端**：轻量设备代理——① 经 tailnet 注册到 server；② 做轻任务/数据采集（如取图、轻预处理或仅中继）；③ **不跑重 torch 模型**；④ 展示结果。MVP 可用 **Termux 托管的 device_client "thin 模式"**（能力标志，不加载重算法）；原生 Kotlin 代理列为后续。定义**设备能力档（compute_class: full/thin）**。

讲稿 | 让安卓以瘦客户端接入，只做采集/轻处理、不跑重模型，并给设备打 "全力/轻活" 能力标签。

- **M3.5 能力感知放置** [已具备基础]：扩展大脑放置（plan/smart），尊重设备能力档——瘦设备只接轻任务/采集，重算法不会被分配到瘦设备。**插入点已明确**：本版已有的**健康预筛**（`_healthy_devices`，从候选设备中先剔除不合格者）即是加 `compute_class(full/thin)` 能力过滤的天然位置，工作量较小。

讲稿 | 让派活认设备能力标签，重活不派给瘦设备；插入点就是已有的健康预筛，改动小。

- **M3.6 各平台打包与安装：**每个桌面 OS 一键引导；Android（Termux）安装文档。

讲稿 | 给各系统做一键安装引导和安装文档。

- **M3.7 跨平台 CI/测试矩阵：**回归套件（或子集）在 Win/Linux/macOS 运行；Android 瘦客户端集成测（或模拟）。

讲稿 | 让回归在 Win/Mac/Linux 都跑、安卓瘦端也测，保证各平台都靠谱。